

Technical Documentation – Milestone 3

AI-Powered Root Cause Analysis & Remediation System Using Retrieval -Augmented Generation

AI Smart Bug Analyzer and Fix Advisor

Prepared By: Harshitha Bejjipuram

Internship: Infosys Springboard Internship

1. Introduction

Software debugging is one of the most time-consuming activities in software development. Developers often spend hours identifying the root cause of issues, searching for similar historical bugs, and determining the most appropriate resolution.

Milestone 3 enhances the **AI Smart Bug Analyzer & Fix Advisor** by integrating **Retrieval-Augmented Generation (RAG)** with Large Language Models (LLMs) to automate bug diagnosis and remediation.

The system intelligently retrieves similar historical bugs, predicts the root cause, identifies duplicate issues, generates remediation recommendations, and presents the findings through an interactive dashboard.

2. Objectives

- The primary objectives of Milestone 3 are:
- Develop an AI-powered Root Cause Analysis Agent.
- Detect duplicate bug reports using semantic similarity.
- Generate remediation recommendations based on historical resolutions.
- Analyze application logs and stack traces.
- Predict software severity, priority, and affected module.
- Present all AI-generated insights in a structured dashboard.
- Reduce debugging effort through intelligent automation.
- To automate software bug investigation using AI-driven analysis techniques.
- To improve debugging efficiency by retrieving relevant historical bug reports through Retrieval-Augmented Generation (RAG).
- To assist developers in making faster and more accurate debugging decisions using AI-generated insights.
- To reduce duplicate bug reporting through semantic similarity-based duplicate detection.
- To provide explainable AI outputs with confidence scores and supporting historical evidence.

- To enhance software quality by recommending immediate fixes and long-term remediation strategies.
- To improve developer productivity by minimizing manual effort during bug diagnosis.
- To provide a centralized dashboard that consolidates all AI-generated bug analysis in a structured format.
- To enable intelligent classification of bugs based on severity, priority, affected module, and log analysis.
- To build a scalable AI-powered bug analysis framework that can support future enterprise software development.

3. Scope

The system is designed to assist software developers during the debugging process.

The scope includes:

- Bug submission
- Log file analysis
- Semantic retrieval of historical bugs
- Root cause prediction
- Duplicate bug detection
- AI-generated remediation
- Dashboard visualization

The system does not replace developers but acts as an intelligent debugging assistant.

4. System Overview

The AI Smart Bug Analyzer consists of multiple intelligent agents that work together to analyze software defects.

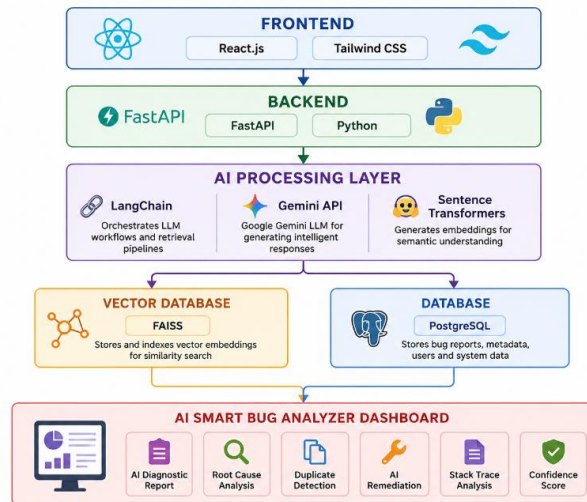
When a developer submits a bug report, the system:

- Extracts bug information.
- Generates embeddings.
- Searches similar bugs using FAISS.
- Retrieves historical resolutions.
- Predicts the root cause.
- Detects duplicate bugs.

- Suggests remediation.
- Displays the results through a dashboard.

5. System Architecture

The architecture consists of the following components:



6. Functional Modules

6.1 Bug Submission Module

The user submits:

- Bug Title
- Description
- Log File
- Stack Trace

The information is stored in the database and forwarded to the AI pipeline.

6.2 AI Bug Triage Module

The Bug Triage module performs the initial analysis.

Functions include:

- Severity Prediction
- Priority Prediction
- Module Prediction

Output

Severity Level

- Priority Level
- Affected Module
- Diagnostic Summary

6.3 Root Cause Analysis Module

The Root Cause Analysis Agent identifies the probable reason for the software failure.

Process

- Convert bug report into embeddings.
- Retrieve similar bugs.
- Send retrieved context to Gemini.
- Generate Root Cause Hypothesis.
- Calculate confidence score.

Output

- Root Cause
- Confidence Score
- Supporting Historical Evidence

6.4 Duplicate Detection Module

This module performs semantic similarity search.

The system retrieves Top-K historical bugs that closely resemble the submitted issue.

Output includes:

- Bug ID
- Similarity Percentage
- Historical Resolution

6.5 AI Remediation Module

The Remediation Agent generates practical recommendations.

Outputs include:

- Immediate Fix
- Long-Term Recommendation
- Engineering Best Practices

6.6 Dashboard Module

The dashboard displays:

- Bug Information
- Severity
- Priority
- Module
- AI Diagnostic Report
- Root Cause
- Duplicate Bugs
- Confidence Score
- Historical Evidence
- Recommended Fix
- Best Practices

7. AI Workflow

Step 1

User submits bug.



Step 2

Bug description and logs are converted into embeddings.



Step 3

FAISS retrieves similar historical bugs.



Step 4

Gemini receives retrieved context.



Step 5

AI predicts:

- Severity
- Priority
- Module
- Root Cause



Step 6

Duplicate Detection retrieves Top-5 similar bugs.



Step 7

AI generates remediation recommendations.



Step 8

Results are displayed in the dashboard.

8. Retrieval-Augmented Generation (RAG)

The application uses RAG to improve the accuracy of AI responses.

The workflow includes:

- Document ingestion
- Embedding generation
- Vector indexing
- Semantic retrieval
- Context augmentation

- LLM response generation

Using historical bugs as context minimizes hallucinations and improves recommendation quality.

9. Functional Requirements

The system shall:

- Accept bug reports.
- Upload log files.
- Parse stack traces.
- Generate embeddings.
- Retrieve similar bugs.
- Predict severity.
- Predict priority.
- Predict affected module.
- Perform root cause analysis.
- Detect duplicate bugs.
- Generate remediation.
- Display structured results.

10. Non-Functional Requirements

- **Performance**
 - Response time should be less than 5 seconds.
- **Scalability**
 - The system should support thousands of historical bug reports.
- **Reliability**
 - The AI pipeline should produce consistent recommendations.
- **Security**
 - Uploaded log files should be securely stored.
- **Maintainability**
 - The application should support future AI enhancements.

11. Technology Stack

Component	Technology
Frontend	React.js
Styling	Tailwind CSS
Backend	FastAPI
Programming Language	Python
Database	PostgreSQL
Vector Database	FAISS
AI Framework	LangChain
LLM	Google Gemini API
Embedding Model	Sentence Transformers

12. Data Flow

1. User submits bug.
2. Backend receives request.
3. Embedding model generates vectors.
4. FAISS retrieves similar bugs.
5. Gemini performs AI analysis.
6. Recommendations are generated.
7. Dashboard displays findings.

13. Database Design

Bugs Table

- Bug ID
- Title
- Description
- Severity

- Priority
- Module
- Resolution

Vector Store

- Embedding ID
- Bug ID
- Vector Representation

14. API Design

- **POST /submitBug**
 - Submit bug details.
- **POST /uploadLogs**
 - Upload application logs.
- **GET /rootCause**
 - Generate root cause analysis.
- **GET /duplicateDetection**
 - Retrieve similar bugs.
- **GET /remediation**
 - Generate AI recommendations.

15. User Interface

- The dashboard contains:
 - Bug Details
 - AI Diagnostic Report
 - Root Cause Analysis
 - Duplicate Detection
 - Stack Trace
 - AI Remediation
 - Historical Evidence

The interface is responsive and optimized for readability.

16. Testing Strategy

- Testing includes:
- Unit Testing
- API Testing
- Integration Testing
- AI Output Validation
- Performance Testing
- User Acceptance Testing

17. Expected Output

- After submitting a bug, the system generates:
- AI Severity Prediction
- AI Priority Prediction
- Module Prediction
- Diagnostic Report
- Root Cause Analysis
- Confidence Score
- Duplicate Bugs
- Historical Evidence
- AI Remediation
- Best Practices

18. Future Enhancements

- Future versions can include:
- Predictive bug prevention
- Real-time CI/CD integration
- Automatic GitHub/Jira issue creation
- Multi-language code analysis

- Explainable AI (XAI)
- Analytics dashboard
- Voice-assisted debugging

19. Conclusion

Milestone 3 significantly enhances the AI Smart Bug Analyzer by introducing intelligent bug triage, semantic retrieval, root cause analysis, duplicate detection, and AI-driven remediation. By leveraging Retrieval-Augmented Generation (RAG), FAISS, Sentence Transformers, LangChain, and Google Gemini, the system provides developers with accurate, context-aware insights that reduce debugging time and improve software quality.